arXiv:2605.18810v1 [cs.LG] 12 May 2026

# D-PACE: Dynamic Position-Aware Cross-Entropy for Parallel Speculative Drafting

**Tianyu Wu**[1] * † **Yu Yao**[2]† **Zhenting Qi**[1] **Han Zheng**[2] **Zhuohan Wang**[1]
**Haoran Ma**[1] **Lawrence Liao**[1] **Himabindu Lakkaraju**[1] **Ju Li**[2]‡ **Yilun Du**[1]‡
[1]Harvard [2]MIT
https://github.com/Lucas-TY/D-PACE

## Abstract

Speculative decoding accelerates LLM inference by having a small drafter propose tokens that a larger target model verifies in parallel. Recent diffusion-based parallel drafters such as DFlash predict the full $B$-token block in one forward pass, enabling deeper drafters and longer accepted blocks. However, existing multi-token drafter objectives often use fixed position-dependent weighting schedules, such as head-dependent weights or block-position decays, which do not adapt as the positions limiting acceptance change during training. To address this, we derive per-position training weights from a differentiable surrogate of expected accepted draft length, matching the weight of each position to its log-probability gradient contribution. The resulting loss, **D-PACE (Dynamic Position-Aware Cross-Entropy)**, shifts training signal toward positions that currently limit acceptance as the drafter improves. Across six benchmarks, two Qwen3-4B draft depths, two decoding temperatures, and two additional target models, D-PACE consistently improves both wall-clock speedup and average emitted length, with 2.3% measured training-time overhead and no changes to the drafter architecture or inference procedure.

## 1 Introduction

Autoregressive decoding is the dominant latency bottleneck of LLM inference because token-by-token generation exposes limited parallelism and often leaves modern GPUs underutilized, especially for the long chains of thought produced by reasoning models [1, 20, 36, 37]. To reduce this sequential bottleneck, speculative decoding (SD) [2, 3, 39] uses a small draft model to propose tokens that a larger target model verifies in parallel.

*Autoregressive drafters with tree decoding*, exemplified by EAGLE [4–6, 9], draft tokens one at a time at inference, with the drafter autoregressively continuing on its own previous outputs. This sequential drafting process limits practical speedups and often constrains practical drafters to shallow architectures under latency budgets [11]. *Parallel single-sequence* methods, building on block-diffusion language models [17, 18] and exemplified by DFlash [11], instead generate all $B$ tokens in a single forward pass, reducing the sequential dependence on block length and allowing deeper, more expressive draft models within the same latency budget.

Parallel block drafting reduces draft-generation latency, but acceptance remains prefix-based: an error at an early position rejects the following draft tokens. Existing objectives therefore often use position-dependent weights. For example, Medusa [7] uses head-dependent loss weights, while DFlash [11] applies a fixed exponential decay within each draft block. These schedules encode prefix importance but depend only on position, rather than on which positions currently limit acceptance

*Project Lead. Correspondence to lucaswu@fas.harvard.edu.
†Equal contribution.
‡Equal advising.

Preprint.

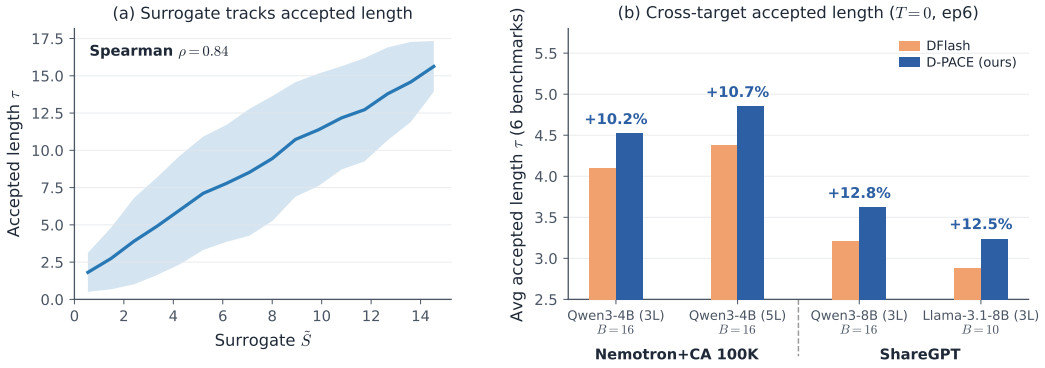



Figure 1: **Surrogate $\tilde{S}$ tracks emitted length $\tau$; D-PACE improves $\tau$ across targets. (a)** Per-block surrogate $\tilde{S} = \sum_{k=1}^{B} \prod_{i \le k} q_i$ ($q_i$: draft confidence on the token selected by the target decoding policy at position $i$) versus reported $\tau$, computed during decoding on the 3L DFlash baseline over 128 MATH-500 prompts; line: bin means; shaded band: $\pm 1\sigma$. Spearman $\rho$=0.84 (App. A). **(b)** Average emitted length $\tau$ over six benchmarks at $T$=0 (epoch 6) for the DFlash baseline and D-PACE across four configurations: Qwen3-4B (3L and 5L, $B$=16), Qwen3-8B (3L, $B$=16), and Llama-3.1-8B (3L, $B$=10). Relative improvement is labeled above each pair.

for a given drafter and example. A fixed schedule therefore misallocates training signal: when early positions are already highly accurate, later positions become the acceptance bottleneck and should receive more weight; when an earlier prediction is incorrect, later positions cannot be accepted regardless of their quality, so training signal assigned to them cannot improve the realized accepted prefix.

To replace the fixed schedule, we seek a training signal that reflects each position's contribution to accepted length. Accepted length is the natural inference-time target, but per-position acceptance is a discrete prefix event determined by the verifier. EAGLE-2 [5] observes at inference that the cumulative product of draft confidences along a path correlates with that path's acceptance rate. In drafters without tree decoding such as DFlash, there is only one such path per block, so this cumulative confidence can serve as a smooth surrogate of the block's expected accepted draft length during training; differentiating it yields each position's contribution under this accepted-length proxy. We empirically verify this on the 3L DFlash baseline (MATH-500): the smooth surrogate $\tilde{S}$ (formally defined in Sec. 3.1) is strongly correlated with reported $\tau$ (Fig. 1; App. A). We use these contributions as per-position weights in our weighted cross-entropy loss, which we call **D-PACE** (Dynamic Position-Aware Cross-Entropy).

**Contributions.** **Adaptive per-position weights.** We derive a weighted cross-entropy objective from a smooth surrogate of expected accepted draft length for parallel single-trajectory drafters. The derivation yields closed-form per-position weights that adapt to which block positions currently limit acceptance for the given drafter and example, replacing DFlash's fixed exponential schedule without changing the drafter architecture or decoding procedure. **Stable training via asymmetric smoothing.** D-PACE stabilizes the cumulative-product weights with asymmetric smoothing: smoothing is applied only when computing the weights, while the cross-entropy term remains on the unsmoothed target-token probability. This preserves token-level CE gradients while preventing early low-confidence positions from suppressing later-position weights. **Evaluation across settings.** On Qwen3-4B, D-PACE improves average decoding speedup by $+8.0\%$–$9.7\%$ and average emitted length by $+8.5\%$–$10.7\%$ across the main Qwen3-4B configurations (Table 1). On Llama-3.1-8B-Instruct and Qwen3-8B, D-PACE improves average emitted length by about $+12.5\%$ and $+12.8\%$, respectively (Table 2).

## 2 Preliminaries

**Speculative decoding notation.** Let $p_\theta$ denote a large *target* model and $q_\psi$ a smaller *draft* model. At each decoding step, the draft generates a block of $B$ candidate tokens $z_1, \ldots, z_B$ from $q_\psi$, and

the target verifies all $B$ tokens in a single forward pass. Verification proceeds left to right: the first position $j$ at which the draft token is rejected terminates the block, and all tokens $z_1, \ldots, z_{j-1}$ before it are accepted. The number of accepted draft tokens per block is the *accepted draft length*

$$X \;=\; \max\big\{k \in \{0, \ldots, B\} : z_i \text{ accepted for all } i \le k\big\}, \tag{1}$$

and, for fixed draft and verification costs, the expected accepted draft length $\mathbb{E}[X]$ is a key factor in the speedup over standard autoregressive decoding [2, 3].

**DFlash block drafter and baseline objective.** DFlash [11] is the parallel block drafter used in our main experiments. Given a verified context, it predicts all $B$ draft positions in one forward pass using a block decoder with KV injection from the target's prefill cache. Let $z_i^*$ denote the token selected by the target decoding policy at block position $i$ and $q_i := q_\psi(z_i^* \mid \text{prefix})$ the draft confidence on that token. The DFlash decayed-CE baseline trains the drafter with

$$\mathcal{L}_{\text{DFlash}} \;=\; \sum_{j=1}^{B} \exp\!\Big(-\frac{j-1}{\gamma}\Big)\,(-\log q_j). \tag{2}$$

These fixed coefficients define the baseline that D-PACE replaces in Sec. 3.

# 3 D-PACE: Dynamic Position-Aware Cross-Entropy

## 3.1 Accepted-Length Surrogate

Using the notation from Sec. 2, let $a_i$ be the per-position acceptance rate, i.e., the probability that position $i$ is accepted given that the first $i{-}1$ positions are accepted. By the chain rule of probability, $\mathbb{P}(X \ge k) = \prod_{i \le k} a_i$. Combined with the tail-sum identity $\mathbb{E}[X] = \sum_{k \ge 1} \mathbb{P}(X \ge k)$ for non-negative integer $X$, this gives

$$\mathbb{E}[X] \;=\; \sum_{k=1}^{B} \mathbb{P}(X \ge k) \;=\; \sum_{k=1}^{B} \prod_{i=1}^{k} a_i. \tag{3}$$

The acceptance rates $\{a_i\}$ in Eq. (3) are not available during training; we use the draft confidence $q_i$ as a smooth proxy for $a_i$ (supported by Fig. 1) to obtain the closed-form surrogate

$$\mathbb{E}[X] \;\approx\; \tilde{S}(\psi) \;:=\; \sum_{k=1}^{B} \prod_{i=1}^{k} q_i, \tag{4}$$

which we use as a smooth proxy for deriving per-position training weights.

**Algorithm 1** D-PACE Optimization

```
Input: draft confidences
       q_i = q_ψ(z_i^* | prefix)

for i = 1, ..., B:
    # smoothing
    q̃_i = (1 − α) · q_i + α

for j = 1, ..., B:
    # prefix products
    P_j = ∏_{i=1}^{j} q̃_i

for j = 1, ..., B:
    # suffix sums of prefix products
    w̄_j = sg(∑_{m=j}^{B} P_m)

return ∑_{j=1}^{B} w̄_j · (− log q_j)
```

Fig. 2 illustrates the difference between DFlash's fixed decay and D-PACE's adaptive weights.

## 3.2 Per-Position Coefficient

To obtain a per-position weight, we differentiate the block-level surrogate $\tilde{S}$ with respect to each $q_j$. For any $j \in \{1, \ldots, B\}$, we split the outer sum in Eq. (4) at $k{=}j$ into terms with and without $q_j$:

$$\tilde{S}(\psi) \;=\; \sum_{k=1}^{j-1} \prod_{i=1}^{k} q_i \;+\; \prod_{i<j} q_i \cdot q_j \cdot \Big(1 + \sum_{m=j+1}^{B} \prod_{i=j+1}^{m} q_i\Big) \;=\; \sum_{k=1}^{j-1} \prod_{i=1}^{k} q_i \;+\; \Big(\prod_{i<j} q_i\Big) q_j\, f_j, \tag{5}$$

where $f_j \;:=\; 1 + \sum_{m=j+1}^{B} \prod_{i=j+1}^{m} q_i$ is the continuation value (the accepted-length value contributed by position $j$ and later positions, conditioned on accepting the prefix through $j$). The first sum is constant in $q_j$, so $\partial \tilde{S} / \partial q_j = (\prod_{i<j} q_i) f_j$: the contribution of position $j$ is modulated by the prefix-confidence factor $\prod_{i<j} q_i$. Applying the log-derivative identity $\nabla_\psi q_j \;=\; q_j \nabla_\psi \log q_j$ yields the

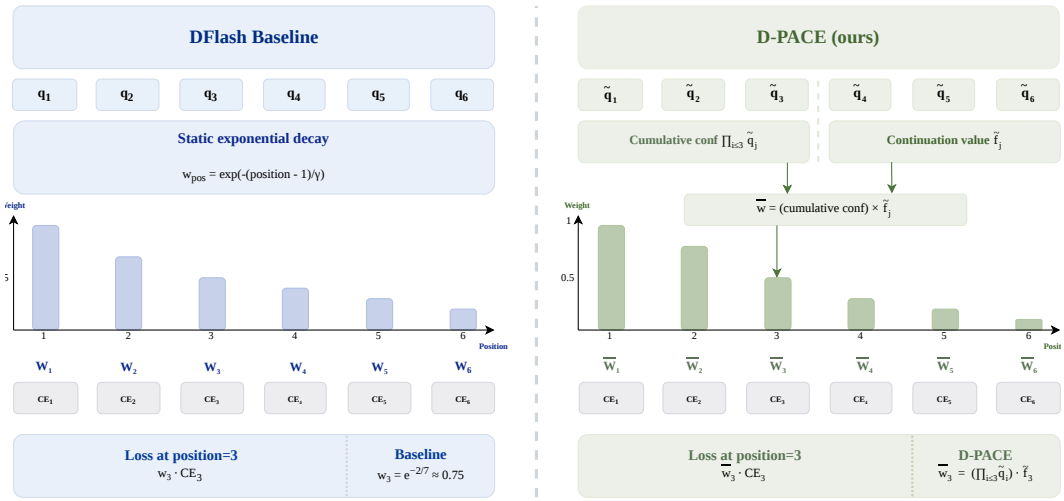



Figure 2: **D-PACE versus the DFlash baseline. Left**: DFlash applies a fixed exponential position decay. **Right**: D-PACE uses example-dependent weights that combine prefix confidence with remaining accepted-length value. Both are weighted cross-entropy objectives with different position weights.

closed-form gradient

$$\nabla_\psi \tilde{S} \;=\; \sum_{j=1}^{B}\Big(\prod_{i<j} q_i\Big) f_j \,\nabla_\psi q_j \;=\; \sum_{j=1}^{B} w_j \,\nabla_\psi \log q_j, \qquad w_j \;:=\; \Big(\prod_{i\le j} q_i\Big) f_j. \tag{6}$$

Since $w_j$ depends only on the draft confidences $\{q_i\}$ in Eq. (6), D-PACE is *draft-only*: after the target-generated training tokens are available, computing the weights requires no additional target-model forward pass. The weight $w_j$ splits into the cumulative confidence $\prod_{i\le j} q_i$ (a proxy for accepting the prefix through position $j$) and the continuation value $f_j$.

### 3.3 Asymmetric Weight Smoothing

The raw weight $w_j = (\prod_{i\le j} q_i) f_j$ is numerically unstable: its cumulative factor $\prod_{i\le j} q_i$ can vanish when multiple small values of $q_i$ are multiplied across positions. To prevent this, we smooth each $q_i$ inside the weight by interpolating toward 1 with mixing parameter $\alpha \in [0, 1]$:

$$\tilde{q}_i \;:=\; (1-\alpha)\, q_i + \alpha. \tag{7}$$

We apply the smoothing in Eq. (7) only inside the weight, leaving $-\log q_j$ unsmoothed so that the cross-entropy retains its standard $1/q_j$ gradient. With $\tilde{q}_i \ge \alpha$, the cumulative product through position $j$ is at least $\alpha^j$, so later weights cannot vanish. Let $\tilde{f}_j$ denote the smoothed analog of $f_j$ (i.e., $f_j$ with $q_i \to \tilde{q}_i$). Applying the same substitution to every $q_i$ in $w_j = (\prod_{i\le j} q_i) f_j$ yields the per-position weight

$$\bar{w}_j \;:=\; \Big(\prod_{i\le j} \tilde{q}_i\Big)\, \tilde{f}_j \;=\; \Big(\prod_{i\le j} \tilde{q}_i\Big)\Big(1 + \sum_{m=j+1}^{B} \prod_{i=j+1}^{m} \tilde{q}_i\Big) \;=\; \sum_{m=j}^{B} \prod_{i\le m} \tilde{q}_i. \tag{8}$$

Substituting $\bar{w}_j$ as the per-position weight in a weighted cross-entropy gives the D-PACE loss

$$\mathcal{L}_{\text{D-PACE}}(\psi) \;=\; \sum_{j=1}^{B} \bar{w}_j \,(-\log q_j), \tag{9}$$

where $\bar{w}_j$ is detached from the gradient (treated as a constant). The weights therefore provide credit assignment for the cross-entropy terms without introducing gradients that decrease a position's own accepted-length contribution. Alg. 1 computes the loss in Eq. (9) from the smoothed confidences, their prefix products, and the corresponding suffix sums in Eq. (8).

# 4 Experiments

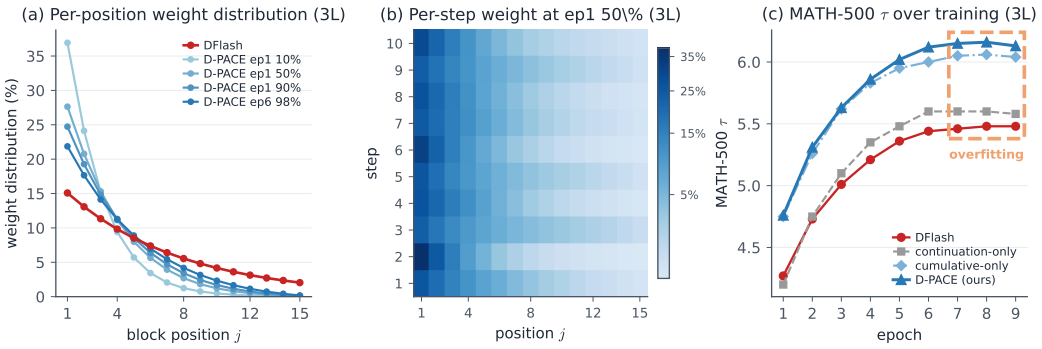



Figure 3: **Weight dynamics and reported emitted length under D-PACE. (a)** On 3L DFlash drafts of Qwen3-4B, mean per-position weight averaged over 100 training steps at four checkpoints (10%, 50%, 90% of epoch 1, and end of epoch 6): DFlash's exponential decay (red) versus D-PACE's per-position weights across checkpoints (blue). **(b)** Per-step weight over 10 consecutive training steps at epoch 1 (50%) (Sec. 5.5). **(c)** MATH-500 reported $\tau$ over training (3L draft, $T$=0) for the DFlash baseline, D-PACE, and its two component-ablation variants (continuation-only, cumulative-only) (Sec. 5.2).

In this section, we conduct experiments to address the following research questions:

- **RQ1 (Efficacy):** How does D-PACE compare with related baselines?
- **RQ2 (Ablation):** How do D-PACE's cumulative-confidence and continuation-value factors affect performance?
- **RQ3 (Generalizability):** Does D-PACE transfer from Qwen3-4B to other target models?
- **RQ4 (Sensitivity):** How sensitive is D-PACE to smoothing and block-size choices?

## 4.1 Experimental Setup

**Models and Benchmarks.** For **Qwen3-4B** [22], we train 3L and 5L DFlash drafters, where 3L/5L denotes the number of draft Transformer layers. All reported drafters use three target hidden-state features. We evaluate the Qwen3-4B drafters at $T = 0$ and $T = 1$. For cross-model generalization (Sec. 5.1), we additionally train 3L drafters for **Llama-3.1-8B-Instruct** [21] and **Qwen3-8B** [22]. We evaluate on six benchmarks: **Math** (GSM8K [24], MATH-500 [38, 27]), **Code** (HumanEval [25], MBPP [26]), and **Chat** (MT-Bench [23], Alpaca [28]). For each benchmark, we report average emitted length $\tau$ and wall-clock speedup over autoregressive decoding (SR); higher is better for both. Training uses a single NVIDIA H200 GPU; SR is measured on an L40S. For Qwen3 targets, we disable thinking mode during evaluation, following DFlash.

**Datasets.** For Qwen3-4B (main experiments), we use the Qwen3-4B-Instruct-100K dataset released by the DFlash authors [11], whose data mixture is based on Nemotron-Post-Training-Dataset-v2 [13] and CodeAlpaca [14]. The dataset provides prompts with target-generated responses for drafter training, and we use it directly to match the released DFlash training data setting. For Llama-3.1-8B-Instruct and Qwen3-8B, we use ShareGPT [15] and generate target-model responses for drafter training, following the EAGLE series [4–6].

**Implementation.** Training is implemented in SpecForge [47], the SGLang team's public speculative-decoding training framework, which provides a third-party implementation of DFlash. Evaluation uses the native benchmark code released by the DFlash authors with the same decoding settings for all methods. We follow DFlash's [11] block-size defaults for each target: $B$=16 for Qwen3-4B and Qwen3-8B, $B$=10 for Llama-3.1-8B. Training uses AdamW (learning rate $6\times10^{-4}$, effective batch size 4: micro-batch 2 $\times$ accumulation 2); full hyperparameters are in App. C. Unless otherwise stated, D-PACE uses smoothing $\alpha = 0.5$ and results use the epoch-6 checkpoint.

Table 1: **Main result.** Decoding speedup (SR) and average emitted length ($\tau$) on six benchmarks at two draft depths (3L, 5L) and two temperatures ($T$=0, 1) for DFlash drafts of Qwen3-4B at epoch 6. "Top-3 prefix mask" is the hard-mask baseline adapted from GRIFFIN [10]. "Accept-rate" directly optimizes the accepted-length surrogate in Eq. 4; see Sec. 4.1 and App. B.

| | | MATH | | | | CODE | | | | CHAT | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | GSM8K | | MATH-500 | | HumanEval | | MBPP | | MT-Bench | | Alpaca | | Avg. | |
| Depth | Method | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ |
| *Temperature $T = 0$* | | | | | | | | | | | | | | | |
| 3L | DFlash | 3.40 | 4.57 | 3.86 | 5.44 | 3.38 | 4.66 | 3.20 | 4.31 | 2.03 | 3.22 | 1.72 | 2.41 | 2.93 | 4.10 |
| | Top-3 prefix mask | 3.29 | 4.44 | 3.67 | 5.19 | 3.22 | 4.43 | 3.06 | 4.13 | 1.95 | 3.11 | 1.62 | 2.29 | 2.80 | 3.93 |
| | Accept-rate | 0.76 | 1.01 | 0.75 | 1.01 | 0.77 | 1.03 | 0.79 | 1.03 | 0.76 | 1.04 | 0.79 | 1.04 | 0.77 | 1.03 |
| | **D-PACE** | **3.75** | **5.09** | **4.32** | **6.12** | **3.71** | **5.12** | **3.47** | **4.68** | **2.16** | **3.54** | **1.79** | **2.54** | **3.20** | **4.52** |
| 5L | DFlash | 3.52 | 4.98 | 3.96 | 5.85 | 3.45 | 4.98 | 3.24 | 4.56 | 2.01 | 3.43 | 1.68 | 2.49 | 2.98 | 4.38 |
| | Top-3 prefix mask | 3.36 | 4.80 | 3.77 | 5.56 | 3.27 | 4.73 | 3.06 | 4.31 | 1.92 | 3.29 | 1.58 | 2.34 | 2.83 | 4.17 |
| | Accept-rate | 0.74 | 1.03 | 0.73 | 1.03 | 0.73 | 1.02 | 0.74 | 1.02 | 0.71 | 1.01 | 0.74 | 1.01 | 0.73 | 1.02 |
| | **D-PACE** | **3.91** | **5.54** | **4.47** | **6.61** | **3.81** | **5.54** | **3.53** | **4.99** | **2.18** | **3.79** | **1.73** | **2.61** | **3.27** | **4.85** |
| *Temperature $T = 1$* | | | | | | | | | | | | | | | |
| 3L | DFlash | 3.23 | 4.37 | 3.41 | 4.91 | 3.16 | 4.37 | 3.04 | 4.11 | 1.93 | 3.05 | 1.67 | 2.32 | 2.74 | 3.86 |
| | Top-3 prefix mask | 3.13 | 4.23 | 3.20 | 4.59 | 3.04 | 4.19 | 2.88 | 3.88 | 1.85 | 2.94 | 1.60 | 2.25 | 2.62 | 3.68 |
| | Accept-rate | 0.76 | 1.01 | 0.74 | 1.01 | 0.77 | 1.04 | 0.78 | 1.03 | 0.75 | 1.04 | 0.80 | 1.04 | 0.77 | 1.03 |
| | **D-PACE** | **3.54** | **4.79** | **3.70** | **5.41** | **3.48** | **4.83** | **3.26** | **4.38** | **2.04** | **3.30** | **1.73** | **2.45** | **2.96** | **4.19** |
| 5L | DFlash | 3.32 | 4.71 | 3.43 | 5.22 | 3.25 | 4.71 | 3.08 | 4.32 | 1.92 | 3.23 | 1.62 | 2.39 | 2.77 | 4.10 |
| | Top-3 prefix mask | 3.20 | 4.54 | 3.24 | 4.91 | 3.06 | 4.43 | 2.86 | 4.01 | 1.80 | 3.03 | 1.51 | 2.23 | 2.61 | 3.86 |
| | Accept-rate | 0.74 | 1.03 | 0.73 | 1.03 | 0.73 | 1.02 | 0.74 | 1.02 | 0.70 | 1.01 | 0.74 | 1.01 | 0.73 | 1.02 |
| | **D-PACE** | **3.72** | **5.31** | **3.80** | **5.73** | **3.58** | **5.19** | **3.33** | **4.73** | **2.03** | **3.49** | **1.71** | **2.56** | **3.03** | **4.50** |

**Baselines.** We compare D-PACE against the **DFlash decayed-CE baseline** (fixed exponential position decay [11]) and **Top-3 prefix mask**, a hard 0/1 CE mask adapted from GRIFFIN [10]: the loss at position $j$ is included only when every earlier target token lies in the draft's top-3 predictions. We also include **Accept-rate**, which directly optimizes the accepted-length surrogate in Eq. 4 and uses neither the detached weighted-CE construction nor asymmetric weight smoothing. We adopt only the GRIFFIN mask construction, not the full GRIFFIN method; all baselines are formalized in App. B.

### 4.2 Main Results

For Qwen3-4B, D-PACE improves both SR and $\tau$ over the DFlash decayed-CE baseline for every setting reported in Table 1. On 3L MATH-500, D-PACE follows a similar convergence trajectory while maintaining a roughly 12% higher $\tau$ than the baseline throughout the 9-epoch run (Fig. 3c).

The related baselines reveal two complementary failure modes. Top-3 prefix mask is too coarse: its hard 0/1 mask discards graded position-wise credit. Accept-rate directly optimizes the surrogate, coupling token likelihood with each position's accepted-length contribution in the same gradient path. It also lacks asymmetric smoothing, allowing early low-confidence positions to suppress later signals. D-PACE separates these roles by computing smoothed weights from the surrogate while retaining token-level CE gradients.

## 5 Ablations and Additional Analyses

### 5.1 Generalization to Other Targets

To test whether the improvement transfers beyond Qwen3-4B, we repeat the 3L training setup on two additional target models: Llama-3.1-8B-Instruct and Qwen3-8B, using the ShareGPT [15] dataset and DFlash's per-target default block sizes ($B$=10 for Llama-3.1-8B, $B$=16 for Qwen3-8B).

D-PACE improves both $\tau$ and SR on every benchmark for both targets (Table 2).

Table 2: **Generalization to other targets.** Per-benchmark SR / $\tau$ at $T$=0 for 3L drafters of Llama-3.1-8B-Instruct and Qwen3-8B (epoch 6).

| Target | Method | GSM8K | MATH-500 | HumanEval | MBPP | MT-Bench | Alpaca |
|---|---|---|---|---|---|---|---|
| Llama-3.1-8B | DFlash | 2.21 / 2.88 | 2.17 / 2.89 | 2.53 / 3.31 | 2.53 / 3.28 | 1.86 / 2.67 | 1.67 / 2.26 |
| | **D-PACE** | **2.52 / 3.29** | **2.47 / 3.28** | **2.82 / 3.69** | **2.84 / 3.68** | **2.08 / 3.00** | **1.83 / 2.49** |
| Qwen3-8B | DFlash | 2.85 / 3.85 | 2.64 / 3.62 | 2.41 / 3.28 | 2.40 / 3.22 | 1.91 / 2.86 | 1.71 / 2.40 |
| | **D-PACE** | **3.25 / 4.40** | **3.04 / 4.20** | **2.75 / 3.76** | **2.65 / 3.56** | **2.07 / 3.19** | **1.82 / 2.60** |

Table 3: **Component ablation on MATH-500 (3L).** MATH-500 $\tau$ over training epochs for the DFlash baseline, D-PACE, and its two component-ablation variants (continuation-only, cumulative-only).

| Epoch | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| DFlash (decayed-CE) | 4.27 | 4.73 | 5.01 | 5.21 | 5.36 | 5.44 | 5.46 | 5.48 | 5.48 |
| continuation-only | 4.20 | 4.75 | 5.10 | 5.35 | 5.48 | 5.60 | 5.60 | 5.60 | 5.58 |
| cumulative-only | 4.75 | 5.26 | 5.62 | 5.83 | 5.95 | 6.00 | 6.05 | 6.06 | 6.04 |
| **D-PACE (full)** | **4.76** | **5.31** | **5.63** | **5.86** | **6.02** | **6.12** | **6.15** | **6.16** | **6.13** |

### 5.2 Component Ablation

To analyze the contribution of each factor, we test two variants: *cumulative-only* ($\bar{w}_j = \prod_{i \le j} \tilde{q}_i$) and *continuation-only* ($\bar{w}_j = \tilde{f}_j$). Both keep the same smoothing as D-PACE. Table 3 shows two patterns.

**(1) The continuation value alone gives limited improvement.** The continuation-only variant drops the cumulative factor $\prod_{i \le j} \tilde{q}_i$ and uses $\tilde{f}_j$ as the weight. It therefore emphasizes positions with large remaining continuation value, but ignores whether the prefix reaches position $j$ with high probability. After the first few epochs, the variant remains about $2\%$ above DFlash in $\tau$ (Table 3), but stays $7\%$–$9\%$ below cumulative-only and D-PACE.

**(2) Prefix acceptance provides the dominant signal.** Cumulative-only weights position $j$ by $\prod_{i \le j} \tilde{q}_i$, a smoothed proxy for the probability of accepting the prefix through position $j$. The cumulative-only variant exceeds DFlash by $10\%$–$12\%$ in $\tau$ at every epoch (Table 3).

Full D-PACE combines both factors: the cumulative term estimates whether the prefix reaches position $j$, while $\tilde{f}_j$ estimates the remaining accepted-length value from position $j$ onward. This combined weighting yields a smaller but consistent improvement over cumulative-only across epochs (Table 3).

### 5.3 Hyperparameter Sensitivity

**Smoothing Parameter $\alpha$.** The smoothing parameter $\alpha$ (Sec. 3.3) sets a lower bound $\tilde{q}_i \ge \alpha$, preventing the cumulative product in the weight from vanishing. We sweep $\alpha$ on three 3L draft configurations—Qwen3-4B at $B$=16 and $B$=8, and Llama-3.1-8B at $B$=10—and report average $\tau$ at epoch 6 (Table 5). Apart from $\alpha = 0$, where the cumulative product vanishes and $\tau$ stalls near 1.0 across the three configurations, every tested $\alpha$ outperforms the DFlash decayed-CE baseline. At the opposite extreme, $\alpha$=0.9 pushes every smoothed confidence near 1 and removes most of the adaptive signal. Performance is stable across $\alpha \in [0.3, 0.7]$, with $\alpha = 0.3$ and $\alpha = 0.5$ performing comparably.

**Block Size $B$.** We re-train 3L drafts at $B \in \{8, 12, 16\}$, following DFlash's per-block-size settings. D-PACE improves average SR and $\tau$ over the DFlash baseline at every block size. This trend is consistent with the role of adaptive weighting: longer blocks expose more positions whose contribution depends on prefix acceptance, while D-PACE adjusts weights from the current block (Table 4).

Table 4: **Block size sensitivity.** Average SR and $\tau$ over the six benchmarks on Qwen3-4B (3L) at epoch 6, across three block sizes.

| | DFlash | | D-PACE | | Δ (%) | |
|---|---|---|---|---|---|---|
| $B$ | SR | $\tau$ | SR | $\tau$ | SR | $\tau$ |
| 8 | 2.85 | 3.88 | **2.97** | **4.08** | +4.2 | +5.2 |
| 12 | 2.95 | 4.08 | **3.16** | **4.42** | +7.1 | +8.3 |
| 16 | 2.93 | 4.10 | **3.20** | **4.52** | +9.2 | +10.2 |

Table 5: **Weight smoothing hyperparameter $\alpha$.** Average $\tau$ over the six benchmarks at epoch 6 across three 3L draft configurations.

| | D-PACE ($\alpha$) | | | | | DFlash |
|---|---|---|---|---|---|---|
| | 0.0 | 0.3 | **0.5** | 0.7 | 0.9 | decayed-CE |
| Qwen3-4B ($B$=16) | 1.02 | 4.51 | **4.52** | 4.47 | 4.32 | 4.10 |
| Qwen3-4B ($B$=8) | 1.02 | **4.09** | 4.08 | 4.04 | 3.98 | 3.88 |
| Llama-3.1-8B ($B$=10) | 1.03 | **3.26** | 3.24 | 3.18 | 2.99 | 2.88 |

### 5.4 Target-Probability Surrogate

As an analytic variant, we replace the draft-side acceptance proxy $q_\psi(z_i^*)$ in the D-PACE weight with the target probability $p_\theta(z_i^*)$ on the same token. In both SR and $\tau$, this variant remains much closer to the DFlash baseline than to full D-PACE (Table 6). Because $p_\theta(z_i^*)$ is independent of the draft model, it cannot capture which positions currently limit acceptance for the given drafter and example.

Table 6: **Target-probability surrogate.** Average SR and $\tau$ over the six benchmarks on the 3L draft at epoch 6. *D-PACE (target prob)* replaces the draft probability with the target probability on the same token in the D-PACE weight.

| Method | SR | $\tau$ |
|---|---|---|
| DFlash (decayed-CE) | 2.93 | 4.10 |
| D-PACE (target prob) | 3.01 | 4.25 |
| **D-PACE (draft prob)** | **3.20** | **4.52** |

### 5.5 Position-Weight Dynamics

*Over training*, D-PACE assigns larger weights to low-index block positions in the early training stage, then shifts weight toward higher-index positions as those positions become more likely to limit acceptance (Fig. 3a); in contrast, DFlash's fixed exponential decay remains constant. *At each training step*, D-PACE recomputes weights from the current block, so the position weights vary across examples and updates (Fig. 3b).

### 5.6 Training-time overhead

Training-time overhead remains small. D-PACE adds a weight-computation step to each training iteration, but the block-level weights are computed in parallel over all positions. In our measurement on a single H200, this increases per-step time by only 2.3% compared with the DFlash decayed-CE baseline (Table 7).

Table 7: Training wall-clock on a single H200 for the 3L draft ($B$=16). Per-step time averages over 49,993 iterations.

| Method | per epoch | per step |
|---|---|---|
| DFlash (decayed-CE) | 2:24 | 173 ms |
| **D-PACE** | 2:28 | 177 ms |

## 6 Related Work

**Speculative decoding architectures.** Speculative decoding proposes tokens with a draft model and verifies them with the target model [2, 3]. Prior work has primarily improved speculative decoding through drafter architecture and verification topology. Medusa [7] and Hydra [29] attach multiple prediction heads to target-model features. The EAGLE series predicts at the feature level, adding dynamic tree pruning in EAGLE-2 and training-time test over intermediate features in EAGLE-3 [4–6]. Other drafters reuse target signals (GLIDE [40], Kangaroo [32]), introduce recurrence (ReDrafter [30]), or use Jacobi-style iteration (Lookahead [8]). On the verification side, SpecInfer [31] verifies top-$K$ draft trajectories as a tree.

**Parallel and diffusion-based drafters.** Parallel block drafters reduce the sequential dependence on block length by predicting multiple positions in one forward pass [19]. This direction connects to diffusion language models, which generate masked positions through parallel denoising steps, including LLaDA [18], Dream [44], Mercury [45], and block-diffusion models such as BD3-LM [17]. DFlash [11] uses a flash-attention block decoder with KV injection from the target's prefill cache; DART [16] predicts multiple future logits and prunes a draft tree; DiffuSpec [43] uses a pre-trained dLLM as a training-free drafter with causal-consistency path search. D-PACE focuses on this parallel

block-drafting regime, where the training objective determines how learning signal is allocated across positions in the block.

**Acceptance-aware drafter objectives.** Several methods modify drafter training to better match downstream acceptance. GRIFFIN [10] masks position $j$'s cross-entropy when any earlier target falls outside the draft's top-$K$; we adapt this mask as a baseline (Sec. 4.1). SpecDiff-2 [41] introduces streak distillation for diffusion drafters, and OPT-Tree [42] uses expected accepted length as an inference-time objective for draft-tree construction. Concurrent LK losses [12] study acceptance-oriented token-level objectives for speculative sampling. Target-side supervision methods such as HASS, FSPAD, DistillSpec, online speculative decoding, and CORAL [9, 33–35, 46] use verifier signals or online feedback. Closest to our setting, DFlash trains its parallel drafter with a fixed exponential decay over block positions. DART [16] also uses a fixed exponential position decay, applied to KL losses over parallel future logits. D-PACE replaces these static schedules with weights derived from an accepted-length proxy. The resulting weight is detached from the cross-entropy, with asymmetric smoothing applied only to the weight while the cross-entropy term keeps its standard gradient.

## 7 Limitations and Future Work

**Scope.** Our derivation applies to parallel block drafters that generate all draft positions in one forward pass. For sequential drafters, later draft generation depends on earlier draft hidden states, so accumulated draft-state errors can break the fixed-prefix factorization used by our accepted-length surrogate. In addition, we evaluate D-PACE only for lossless speculative decoding. More broadly, contribution-based weighting may be relevant to other parallel block-prediction settings with prefix-truncated utility. Such settings may include multi-token prediction heads and other blockwise sequence-generation objectives, which we do not evaluate here.

**Objective extensions.** Our experiments evaluate D-PACE as a dynamic weighting scheme for token-level cross-entropy training in parallel block drafters such as DFlash. App. E (**D-PAKL**) derives the corresponding KL-distillation variant by replacing the soft acceptance analog $\langle p_i, q_\psi \rangle$ with the Jensen lower-bound proxy $\exp(-C_i)$. Empirical evaluation of D-PAKL and other forms of acceptance-aware dynamic loss weighting remains future work.

**Acceptance surrogate.** D-PACE uses $q_i = q_\psi(z_i^* \mid \text{prefix})$, the draft probability of the token selected by the target decoding policy, as a smooth proxy for whether position $i$ will be accepted. Under hard-match verification, this is not an exact acceptance probability: acceptance is a discrete match event, whereas $q_i$ is the drafter's confidence on the target-selected token. Especially early in training, this confidence proxy may be weakly aligned with realized acceptance; we do not separately evaluate that effect. We rely on the proxy empirically, supported by the correlation between $\tilde{S}$ and reported $\tau$ in Fig. 1 and App. A.

**Broader impact.** D-PACE improves DFlash's acceleration performance without altering the target model's distribution, translating to lower energy consumption and serving cost at scale. It broadly applies to existing models rather than enabling new capabilities, introducing no novel risks beyond those inherent in the underlying LLMs.

## 8 Conclusion

**D-PACE** replaces DFlash's fixed exponential decay with per-position weights derived from the gradient of an accepted-length surrogate, tying the drafter's training signal to accepted length, a key factor in speculative decoding efficiency. With 2.3% training-time overhead (Table 7) and no architectural or inference-pipeline changes, D-PACE improves average emitted length and speedup on DFlash drafts of Qwen3-4B (reaching up to $4.47\times$ speedup on MATH-500; Table 1) and transfers to Llama-3.1-8B and Qwen3-8B (Sec. 5.1). Beyond DFlash, the same construction may apply to other single-trajectory drafters when a differentiable proxy for acceptance is available. More broadly, D-PACE suggests that multi-token drafters can replace fixed position schedules with weights derived from how each position contributes to accepted length.

## References


[1] J. Achiam et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proc. ICML*, 2023.

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proc. ICML*, 2024.

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proc. EMNLP*, 2024.

[6] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Proc. NeurIPS*, 2025.

[7] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proc. ICML*, 2024.

[8] Y. Fu, P. Bailis, I. Stoica, and H. Zhang. Break the sequential dependency of LLM inference using lookahead decoding. In *Proc. ICML*, 2024.

[9] L. Zhang, X. Wang, Y. Huang, and R. Xu. Learning harmonized representations for speculative sampling (HASS). In *Proc. ICLR*, 2025.

[10] S. Hu, J. Li, X. Xie, Z. Lu, K.-C. Toh, and P. Zhou. GRIFFIN: Effective token alignment for faster speculative decoding. In *Proc. NeurIPS*, 2025.

[11] J. Chen, Y. Liang, and Z. Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[12] A. Samarin, S. Krutikov, A. Shevtsov, S. Skvortsov, F. Fisin, and A. Golubev. LK losses: Direct acceptance rate optimization for speculative decoding. *arXiv preprint arXiv:2602.23881*, 2026.

[13] D. Nathawani, S. Ding, V. Lavrukhin, I. Gitman, S. Majumdar, E. Bakhturina, B. Ginsburg, and J. Polak Scowcroft. Nemotron-Post-Training-Dataset-v2. Hugging Face dataset, 2025. https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v2.

[14] S. Chaudhary. Code Alpaca: An instruction-following LLaMA model for code generation. GitHub repository, 2023. https://github.com/sahil280114/codealpaca.

[15] ShareGPT. ShareGPT: GPT-4/ChatGPT conversations shared by users. Hugging Face dataset (Aeala re-upload), 2023. https://huggingface.co/datasets/Aeala/ShareGPT_Vicuna_unfiltered.

[16] F. Liu, X. Li, K. Zhao, Y. Gao, Z. Zhou, Z. Zhang, Z. Wang, W. Dou, S. Zhong, and C. Tian. DART: Diffusion-inspired speculative decoding for fast LLM inference. *arXiv preprint arXiv:2601.19278*, 2026.

[17] M. Arriola et al. Block diffusion: Interpolating between autoregressive and diffusion language models. In *Proc. ICLR*, 2025.

[18] S. Nie et al. Large language diffusion models (LLaDA). *arXiv preprint arXiv:2502.09992*, 2025.

[19] M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Proc. NeurIPS*, 2018.

[20] H. Touvron et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[21] A. Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[22] A. Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[23] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Proc. NeurIPS Datasets & Benchmarks*, 2023.

[24] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[25] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[26] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[27] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proc. NeurIPS Datasets & Benchmarks*, 2021.

[28] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. GitHub repository, 2023. https://github.com/tatsu-lab/stanford_alpaca.

[29] Z. Ankner, R. Parthasarathy, A. Nrusimha, C. Rinard, J. Ragan-Kelley, and W. Brandon. Hydra: Sequentially-dependent draft heads for Medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.

[30] Y. Cheng, A. Zhang, X. Zhang, C. Wang, and Y. Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[31] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia. SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification. In *Proc. ASPLOS*, 2024.

[32] F. Liu, Y. Tang, Z. Liu, Y. Ni, K. Han, and Y. Wang. Kangaroo: Lossless self-speculative decoding via double early exiting. In *Proc. NeurIPS*, 2024.

[33] L. Gui, B. Xiao, L. Su, and W. Chen. Boosting lossless speculative decoding via feature sampling and partial alignment distillation. *arXiv preprint arXiv:2408.15562*, 2024.

[34] Y. Zhou, K. Lyu, A. S. Rawat, A. K. Menon, A. Rostamizadeh, S. Kumar, J.-F. Kagy, and R. Agarwal. DistillSpec: Improving speculative decoding via knowledge distillation. In *Proc. ICLR*, 2024.

[35] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Online speculative decoding. In *Proc. ICML*, 2024.

[36] H. Touvron et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[37] D. Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[38] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *Proc. ICLR*, 2024.

[39] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*, 2024.

[40] C. Du, J. Jiang, Y. Xu, J. Wu, S. Yu, Y. Li, S. Li, K. Xu, L. Nie, Z. Tu, and Y. You. GliDe with a CaPE: A low-hassle method to accelerate speculative decoding. In *Proc. ICML*, 2024.

[41] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto. SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding. *arXiv preprint arXiv:2511.00606*, 2025.

[42] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang. OPT-Tree: Speculative decoding with adaptive draft tree structure. *Trans. Assoc. Comput. Linguist.*, 13:188–199, 2025.

[43] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang. DiffuSpec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358*, 2025.

[44] J. Ye et al. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

[45] Inception Labs, S. Khanna, S. Kharbanda, S. Li, H. Varma, E. Wang, S. Birnbaum, Z. Luo, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

[46] Y. Weng, D. Mei, H. Qiu, X. Chen, L. Liu, J. Tian, and Z. Shi. CORAL: Learning consistent representations across multi-step training with lighter speculative drafter. *arXiv preprint arXiv:2502.16880*, 2025.

[47] S. Li, C. Wang, Y. Zhu, Y. Wang, F. Yin, S. Shi, Y. Chen, X. Dong, Q. Chen, J. Pan, J. Li, L. Xie, Y. Zhang, L. Yu, Y. Wen, I. Tsang, and T. Zhang. SpecForge: A flexible and efficient open-source training framework for speculative decoding. *arXiv preprint arXiv:2603.18567*, 2026.

# A Confidence–acceptance correlation

We compute the per-block Spearman rank correlation between two confidence statistics and reported $\tau$ over all blocks from the 3L DFlash baseline ($B$=16) on 128 MATH-500 prompts. For a confidence statistic $s_b$ computed on block $b$, we compute

$$\rho \;=\; \mathrm{corr}(\mathrm{rank}(s_b),\, \mathrm{rank}(\tau_b))\,.$$

Table 8 compares the surrogate $\tilde{S} = \sum_{k=1}^{B} \prod_{i \le k} q_i$ used in Sec. 3.1 against the plain confidence sum $\sum_i q_i$.

Table 8: Per-block Spearman rank correlation between reported $\tau_b$ and two block-level confidence statistics on 128 MATH-500 prompts. $\sum_i q_i$ sums per-position draft confidence, while $\tilde{S} = \sum_{k=1}^{B} \prod_{i \le k} q_i$ sums cumulative prefix-confidence terms, where $k$ indexes the accepted-prefix length. Both correlations have $p < 10^{-10}$.

| Per-block statistic | Spearman $\rho$ |
|---|---|
| $\sum_i q_i$ | 0.79 |
| $\tilde{S} = \sum_k \prod_{i \le k} q_i$ (Fig. 1) | **0.84** |

The cumulative surrogate $\tilde{S}$ is more predictive of reported $\tau$ than the plain confidence sum.

# B Loss formulations

Let $B$ be the block size, $z_i^*$ the target token at position $i$, $q_i = q_\psi(z_i^* \mid \text{prefix})$ the draft's confidence on $z_i^*$, $\tilde{q}_i = (1-\alpha)q_i + \alpha$ the smoothed confidence (default $\alpha = 0.5$), and $\tilde{f}_j = 1 + \sum_{m=j+1}^{B} \prod_{i=j+1}^{m} \tilde{q}_i$ the smoothed continuation value (Sec. 3.3).

## B.1 D-PACE

This is the main D-PACE loss from Sec. 3.3:

$$\mathcal{L}_{\text{D-PACE}} \;=\; \sum_{j=1}^{B} \bar{w}_j\,(-\log q_j), \qquad \bar{w}_j \;=\; \Big(\prod_{i \le j} \tilde{q}_i\Big)\tilde{f}_j.$$

## B.2 Baselines

**DFlash decayed-CE (Sec. 4.1).** The released DFlash baseline applies an exponential position decay [11] with a block-size-specific decay constant:

$$\mathcal{L}_{\text{DFlash}} \;=\; \sum_{j=1}^{B} \exp\!\Big(-\frac{j-1}{\gamma_B}\Big)\,(-\log q_j).$$

DFlash sets $\gamma_B$ to 7, 5, and 4 for block sizes 16, 10, and 8, respectively; for $B$=12, we use $\gamma_{12} = 6$.

**Top-$K$ prefix mask (Sec. 4.1).** Adapted from GRIFFIN [10]; we use $K$=3 and adopt only this mask, not the full GRIFFIN method:

$$\mathcal{L}_{\text{top-}K} \;=\; \sum_{j=1}^{B} m_j\,(-\log q_j), \qquad m_j \;=\; \prod_{i<j} \mathbf{1}\big[z_i^* \in \mathrm{Top}_K\, q_\psi(\cdot \mid \text{prefix})\big].$$

Position $j$ contributes only when every earlier target token lies in the draft's top-$K$ predictions.

**Accept-rate (Sec. 4.1).** This baseline directly ascends the accepted-length surrogate in Eq. 4, with no smoothing or detached weighted-CE construction:

$$\mathcal{L}_{\text{accept-rate}} \;=\; -\sum_{m=1}^{B} \prod_{j=1}^{m} q_j.$$

### B.3 Ablation variants of D-PACE

**Cumulative-only and continuation-only (Sec. 5.2).** Substituting each factor of $\bar{w}_j$ separately into $\mathcal{L}_{\text{D-PACE}}$:

$$\mathcal{L}_{\text{cumulative-only}} = \sum_{j=1}^{B} \Big(\prod_{i\le j} \tilde{q}_i\Big)\,(-\log q_j),$$

$$\mathcal{L}_{\text{continuation-only}} = \sum_{j=1}^{B} \tilde{f}_j\,(-\log q_j).$$

**Target-probability surrogate (Sec. 5.4).** This variant replaces each $q_\psi(z_i^*)$ inside the weight by the target probability $p_\theta(z_i^*)$, while keeping the cross-entropy on the draft:

$$\mathcal{L}_{\text{target-prob}} = \sum_{j=1}^{B} \bar{w}_j^{\text{target}}\,(-\log q_j), \qquad \bar{w}_j^{\text{target}} = \Big(\prod_{i\le j} \tilde{p}_i\Big)\tilde{f}_j^{\text{target}},$$

where $\tilde{p}_i = (1-\alpha)\,p_\theta(z_i^*) + \alpha$ and $\tilde{f}_j^{\text{target}}$ is the analogous continuation value computed from $\tilde{p}_i$.

## C Training details

In addition to the hyperparameters reported in Sec. 4.1, training uses sequence length 2048, warmup ratio 0.04, gradient clipping at max-norm 1.0, a cosine learning-rate schedule after warmup, and 512 anchor positions per sequence. Within each paired comparison, only the training objective differs; across reported configurations, draft depth, target model, and block size vary as stated in Sec. 4.1.

**Loss formulations.** All training losses—D-PACE, the DFlash decayed-CE baseline, the Top-$K$ prefix mask, Accept-rate, and the ablation variants—are consolidated in App. B.

## D Training compute

All training runs use a single NVIDIA H200 GPU. A 3L draft run takes approximately 2:25 per epoch and is trained for 6 epochs, giving roughly 15 H200 GPU-hours per experiment setting. Across the main comparisons, baseline runs, and ablations reported in the paper, total training compute is approximately 465 H200 GPU-hours. Table 7 reports the wall-clock comparison used for the overhead estimate.

## E D-PAKL: KL-distillation variant

For drafters trained with a KL distillation loss against a soft target distribution $p_i := p_\theta(\cdot \mid \text{prefix})$ in place of hard cross-entropy on a sampled token $z_i^*$, define the soft-label cross-entropy

$$C_i := -\sum_x p_i(x)\,\log q_\psi(x).$$

The KL loss satisfies $\mathrm{KL}(p_i \,\|\, q_\psi) = C_i - H(p_i)$, so $C_i$ and the KL have the same gradient with respect to the draft parameters. The hard-CE setting in Sec. 3 is the special case $p_i = \delta_{z_i^*}$.

### E.1 Accepted-length surrogate

In Sec. 3.1, the hard-label surrogate uses the draft confidence $q_i = q_\psi(z_i^* \mid \text{prefix})$ as the proxy for the per-position acceptance rate $a_i$ in $\mathbb{E}[X] = \sum_{k=1}^{B}\prod_{i=1}^{k} a_i$. With a soft target, no single sampled token $z_i^*$ exists; the analog of $q_i$ is the inner product $\langle p_i, q_\psi\rangle = \sum_x p_i(x)\,q_\psi(x)$, which collapses to $q_i$ on a Dirac target. By Jensen's inequality on the concave log,

$$\exp(-C_i) = \exp\big(\mathbb{E}_{p_i}[\log q_\psi]\big) \le \mathbb{E}_{p_i}[q_\psi] = \langle p_i, q_\psi\rangle, \tag{10}$$

with equality on a Dirac target. We use the Jensen lower bound as the proxy,

$$\hat{a}_i \;:=\; \exp\!\big(-C_i\big), \tag{11}$$

so that $-\log \hat{a}_i = C_i$ by construction, and $\hat{a}_i$ requires no extra forward computation. The resulting surrogate is

$$\tilde{S}(\psi) \;:=\; \sum_{k=1}^{B}\prod_{i=1}^{k}\hat{a}_i. \tag{12}$$

### E.2 Per-position coefficient

Differentiating Eq. (12) with respect to each $\hat{a}_j$ as in Sec. 3.2, split the outer sum at $k = j$:

$$\tilde{S} \;=\; \sum_{k=1}^{j-1}\prod_{i=1}^{k}\hat{a}_i \;+\; \Big(\prod_{i<j}\hat{a}_i\Big)\,\hat{a}_j\,\Big(1+\sum_{m=j+1}^{B}\prod_{i=j+1}^{m}\hat{a}_i\Big) \;=\; \sum_{k=1}^{j-1}\prod_{i=1}^{k}\hat{a}_i \;+\; \Big(\prod_{i<j}\hat{a}_i\Big)\,\hat{a}_j\,f_j,$$

where the continuation value is

$$f_j \;:=\; 1+\sum_{m=j+1}^{B}\prod_{i=j+1}^{m}\hat{a}_i.$$

The first sum is constant in $\hat{a}_j$, so $\partial\tilde{S}/\partial\hat{a}_j = (\prod_{i<j}\hat{a}_i)\,f_j$. Applying $\nabla_\psi \hat{a}_j = \hat{a}_j\,\nabla_\psi \log \hat{a}_j$ and using $-\log \hat{a}_j = C_j$,

$$\nabla_\psi \tilde{S} \;=\; \sum_{j=1}^{B}\Big(\prod_{i\le j}\hat{a}_i\Big)\,f_j\,\nabla_\psi \log \hat{a}_j \;=\; -\sum_{j=1}^{B} w_j\,\nabla_\psi C_j, \qquad w_j \;:=\; \Big(\prod_{i\le j}\hat{a}_i\Big)\,f_j. \tag{13}$$

### E.3 Weight smoothing

The cumulative product $\prod_{i\le j}\hat{a}_i$ vanishes when several $\hat{a}_i$ are small. Following Sec. 3.3, we smooth each $\hat{a}_i$ inside the weight only, leaving $C_j$ unsmoothed:

$$\tilde{a}_i \;:=\; (1-\alpha)\,\hat{a}_i + \alpha, \qquad \tilde{f}_j \;:=\; 1+\sum_{m=j+1}^{B}\prod_{i=j+1}^{m}\tilde{a}_i, \qquad \bar{w}_j \;:=\; \Big(\prod_{i\le j}\tilde{a}_i\Big)\,\tilde{f}_j.$$

### E.4 D-PAKL loss

Substituting $\bar{w}_j$ as the per-position weight gives the D-PAKL loss

$$\mathcal{L}_{\text{D-PAKL}}(\psi) \;=\; \sum_{j=1}^{B}\bar{w}_j\,C_j,$$

with $\bar{w}_j$ detached from the gradient. Equivalently, one may replace $C_j$ by $\mathrm{KL}(p_j\|q_\psi)$ in the final weighted loss because $H(p_j)$ is constant with respect to the draft parameters. In the hard-CE limit, Eq. (10) is tight, $\hat{a}_i = q_i$, $C_j = -\log q_j$, and D-PAKL reduces to D-PACE (Eq. 9).